



TaskFlow

Architektura, decyzje i wnioski

Projektowanie Aplikacji Internetowych

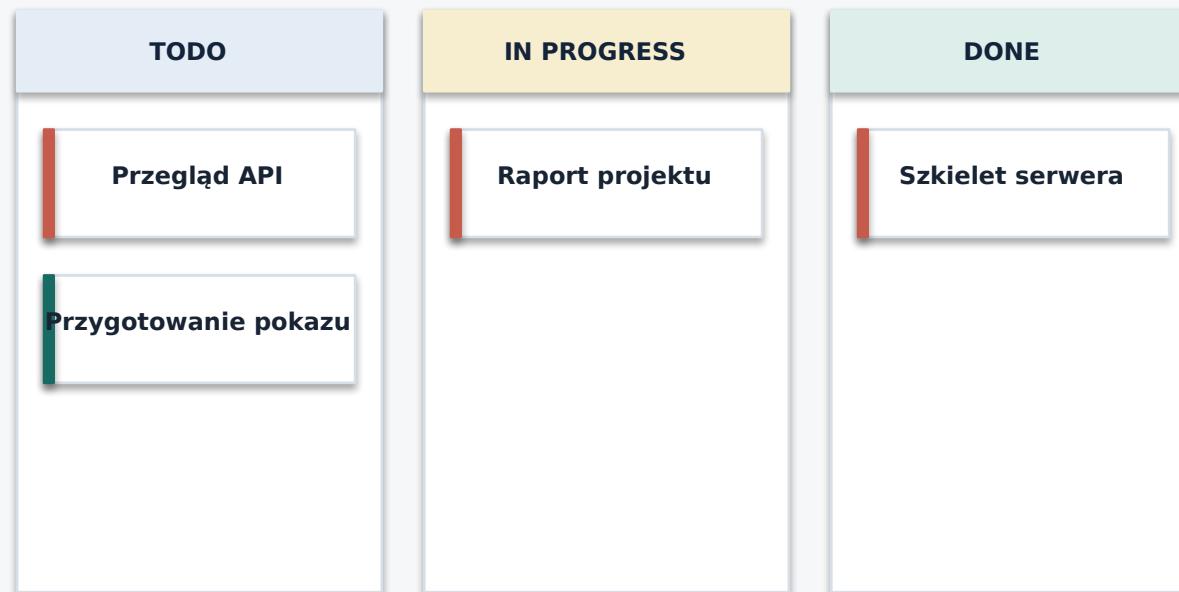
Jakub Uchman | Marcin Kuduk

Vue 3 | Spring Boot | PostgreSQL

Co robi TaskFlow?

Lekki system zarządzania projektami i zadaniami w stylu kanban

- Rejestracja, logowanie i chroniona sesja JWT
- Projekty i zadania z pełnym CRUD
- Tablica kanban z przeciąganiem kart
- Notatki, statystyki, raport i sugestia zadania

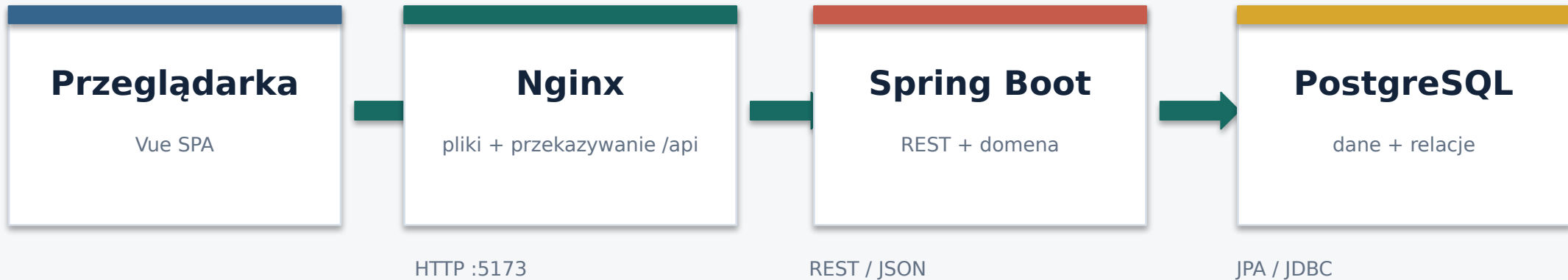


DEMONSTRACJA

logowanie > projekt > przesunięcie > notatka > raport

Jeden system, trzy kontenery

Modularny monolit i powtarzalne uruchomienie przez Docker Compose



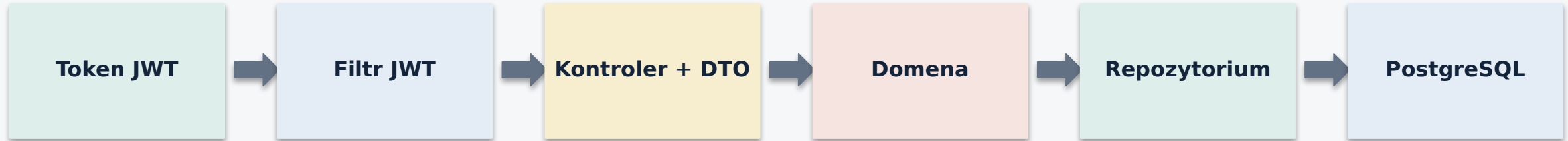
docker compose

Jedna komenda uruchamia interfejs, API i bazę

Nginx przekazuje żądania /api; lokalnie tę samą rolę pełni Vite.

Przepływ chronionego wywołania API

Serwer jest źródłem prawdy dla walidacji, uprawnień i reguł domenowych



Bezpieczeństwo

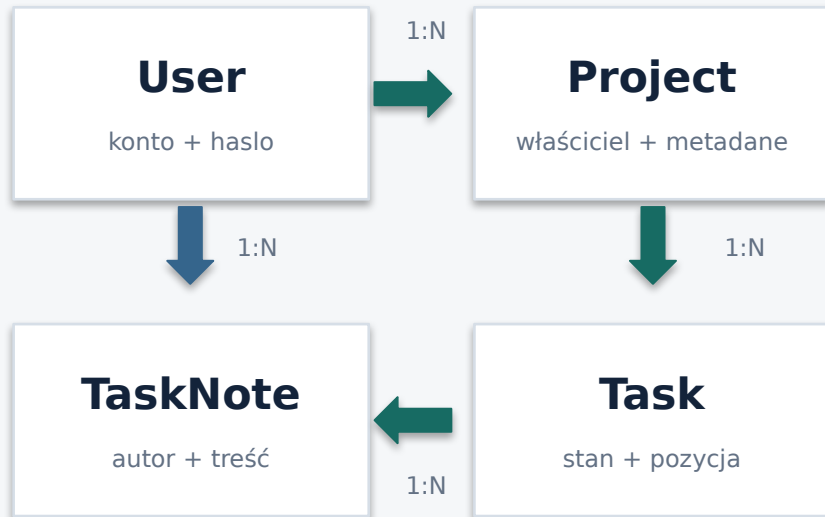
- Repozytorium ogranicza zapytania do właściciela
- Cudzy zasób i brak zasobu zwracają to samo 404

Domena

- Przeniesienie porządkuje obie kolumny w jednej transakcji
- completedAt i raporty są wyznaczane po stronie serwera

Model domenowy i operacje domenowe

Cztery powiązane zasoby, ale logika wykracza poza CRUD



Operacje wynikające z potrzeb interfejsu

GET /board

gotowe kolumny kanban

PATCH /tasks/{id}/move

atomowa zmiana przepływu pracy

GET /stats + /report

jedna definicja metryk

GET /suggested-next-task

jawny ranking, bez AI

Dwie decyzje, które zdefiniowały zakres

Spójny model dostępu i prosta topologia zamiast częściowych rozwiązań

ADR-001

Jeden właściciel zamiast zaproszeń

- Jedna reguła dostępu dla całego grafu zasobów
- Zaproszenia zmieniają dane, API, interfejs i testy
- Rozszerzenie wymaga pełnych ról OWNER / MEMBER

Zmiana roboczego założenia po ocenie kosztu

ADR-002

Modularny monolit

- Jedna transakcja i jeden model uprawnień
- Moduły według domeny, nie warstw technicznych
- Bez kosztu sieci i wdrażania mikroserwisów

Granice w kodzie bez infrastruktury rozproszonej

Stos wynika z problemu, nie z popularności

Alternatywy porównaliśmy według kosztu, ryzyka i potrzeb domeny

REST + operacje

DLACZEGO NIE:

- GraphQL: brak potrzeby wyboru pól
- gRPC: gorsze wsparcie przeglądarki
- tRPC: serwer wymaga TypeScript

PostgreSQL + JPA

DLACZEGO NIE:

- MongoDB: dane są relacyjne
- H2: ryzyko różnic między dialektami
- jOOQ: zbędna kontrola nad SQL

Vue 3 SPA

DLACZEGO NIE:

- React: podobny koszt, brak przewagi
- Angular: zbyt duży narzut
- Nuxt/SSR: brak publicznych treści

Doświadczenie zespołu w Javie i Spring Boot ograniczyło ryzyko, ale nie było jedynym argumentem.

Ryzyka pokrywamy tam, gdzie powstają

Priorytet testów serwera i elementy dodatkowe faktycznie używane w systemie

78

przypadków testowych

10

klas Spocka

5

elementów dodatkowych

18

decyzji ADR

Strategia testów

JEDNOSTKOWE

metryki i ranking sugestii

INTEGRACYJNE

MockMvc + Spring Security + Flyway
+ prawdziwy PostgreSQL

Dodatkowe elementy

Testy

Walidacja DTO

Swagger

Dane próbne

Stan i dziennik

Każde wymaganie ma konkretny dowód

Nie tylko deklarujemy architekturę: wskazujemy jej realizację

R1

API + 4 zasoby

Użytkownik, Projekt, Zadanie, Notatka

R2

Baza + migracje

PostgreSQL, Flyway, ddl-auto=none

R3

Interfejs

Vue SPA korzystające z REST API

R4

Uwierzytelnianie

JWT, BCrypt, chronione API

R5

Kontenery

Compose: interfejs + serwer + baza

R6

Repozytorium

README, ADR, wkład obu osób

Czego się nauczyliśmy?

Najważniejsze efekty nie są związane z liczbą punktów API

ADR może się zmienić

Decyzje mogą ewoluować.
Zaproszenia odroczyliśmy po ocenie wpływu.

01

Granice redukują ryzyko

Bezpieczeństwo pozostaje na serwerze.
Interfejs korzysta z kontraktu API.

02

Testuj na docelowym stosie

PostgreSQL w Testcontainers ujawnia
błędy niewidoczne w H2.

03

Co zrobilibyśmy inaczej?

Konkretna retrospektywa po zakończeniu implementacji

Testy interfejsu



Vitest dla uwierzytelniania i Playwright dla głównego przepływu

Analiza kodu od początku



serwer i interfejs w jednej bramce jakości CI

ADR po każdej zmianie



aktualizacja zakresu razem ze zmianą, nie na końcu

Podział dużego widoku



wydzielenie tablicy, okien dialogowych i sekcji analiz

TASKFLOW

Pytania?

REPOZYTORIUM

github.com/Inpheon/TaskFlow

SWAGGER

localhost:8080/swagger-ui/index.html

DOKUMENTY

[docs/ADR.md](#) + [docs/api-contract.md](#)

Dziękujemy